

MEDIUM-1：位级操作

模块一：启动引导区 —— 语言的本源

【任务 1】

- 1.十进制211对应的二进制表示为：11010011
- 2.二进制10110110对应的十进制表示为182
- 3.判断十进制整数奇偶：用2取余，余数为1的是奇数，为0的是偶数

模块二：能量阵列 —— 逻辑门的重构

【任务 2】

1.

```
1  #include <stdio.h>
2
3  int main() {
4      unsigned char a = 12; // 二进制：00001100
5      unsigned char b = 25; // 二进制：00011001
6
7      unsigned char res1 = a & b;
8      unsigned char res2 = a | b;
9      unsigned char res3 = a ^ b;
10     unsigned char res4 = (a << 2) | (b >> 1);
11
12     printf("%d %d %d %d\n", res1, res2, res3, res4);
13     return 0;
14 }
```

- res1 = 8 (12 (00001100) 和25 (00011001) 对应位都为 1 时才为 1，否则为 0，运算结果为 00001000，换算成十进制数就是8)
- res2 = 29 (12 (00001100) 和25 (00011001) 对应位只要有 1 就为 1，否则为 0，运算结果为 00011101，换算成十进制数就是29)
- res3 = 21 (12 (00001100) 和25 (00011001) 对应位不同为 1，相同为 0，运算结果为00010101，换算成十进制数就是21)
- res4 = 60 (12 (00001100) 各二进位全部左移2位，高位丢弃，低位补0，运算结果为00110000；25 (00011001) 各二进位全部右移1位，用unsigned定义，是无符号数，高位补 0，低位丢弃，所以运算结果为00001100；最后再对两边结果进行 |（按位或）计算，得到00111100，换算成十进制数就是60)

有图有真相：

```
getNew4 > C task2.1.c
1  #include <stdio.h>
2
3  int main() {
4      unsigned char a = 12; // 二进制: 00001100
5      unsigned char b = 25; // 二进制: 00011001
6
7      unsigned char res1 = a & b;
8      unsigned char res2 = a | b;
9      unsigned char res3 = a ^ b;
10     unsigned char res4 = (a << 2) | (b >> 1);
11
12     printf("%d %d %d %d\n", res1, res2, res3, res4);
13     return 0;
14 }
```

问题 输出 调试控制台 终端 端口

```
PS D:\c\vscode\vscode> cd 'd:\c\vscode\vscode\getNew4\output'
● PS D:\c\vscode\vscode\getNew4\output> & .\'task2.1.exe'
● 8 29 21 60
```

2.

对1做“<<”运算，可以标定需要获知的位置，通过“&”运算的返回值来决定输出结果

与此同时也要考虑到输入数字为0，以及查找位数太大或者太小的情况

另外，我还添加了“输出该数字的二进制表示”的功能（同样也是借助位运算），更加直观的判断运行结果是否正确

```
1  #include <stdio.h>
2
3  int main(){
4      int x;
5      int n;
6      printf("请输入一个十进制数");
7      scanf("%d",&x);
8      printf("你要查询该数字的第几位: ");
9      scanf("%d",&n);
10
11
12
13     if (x == 0) {
14         printf("该数字的二进制为: 0\n");
15         return 0;
16     }
17     printf("该数字的二进制为: ");
18
19     int started = 0;
20
21     for (int i = 31; i >= 0; i--) {
22         //从第一个1开始输出
23         if (x & (1 << i)) {
```

```

24         started = 1;
25     }
26     if (started) {
27         printf("%d", (x & (1 << i)) ? 1 : 0);
28     }
29 }
30 //防止出现超位
31 if (n < 1 || n > 31) {
32     printf("\n你输入的位数超出范围");
33     return 1;
34 }
35
36 if(x & (1 << (n-1))){
37     printf("\n该数字的第%d位为: 1",n);
38 }else {
39     printf("\n该数字的第%d位为: 0",n);
40 }
41 return 0;
42 }

```

运行结果如下图:

```

19     int started = 0;
20
21     for (int i = 31; i >= 0; i--) {
22         //从第一个1开始输出
23         if (x & (1 << i)) {
24             started = 1;
25         }
26         if (started) {
27             printf("%d", (x & (1 << i)) ? 1 : 0);
28         }
29     }
30     //防止出现超位
31     if (n < 1 || n > 31) {
32         printf("\n你输入的位数超出范围");
33         return 1;
34     }
35
36     if(x & (1 << (n-1))){
37         printf("\n该数字的第%d位为: 1",n);
38     }else {
39         printf("\n该数字的第%d位为: 0",n);
40     }
41     return 0;

```

问题 输出 调试控制台 终端 端口

```

• PS D:\c\.vscode\.vscode> ${env:DEBUGINFOD_URLS}='' & 'c:\Users\Mark\.vscode
.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-M
Microsoft-MIEngine-Out-d3eq3ddq.bv4' '--stderr=Microsoft-MIEngine-Error-vzqk
Pid-c1f2vqwy.uek' '--dbgExe=D:\c\mingw64\bin\gdb.exe' '--interpreter=mi'
请输入一个十进制数251
你要查询该数字的第几位: 3
该数字的二进制为: 11111011
该数字的第3位为: 0

```

3.

将第n位的值改为修正值t，需要拆解成两部分来解决：

- 当t = 1时，通过“<<”运算确定需要修改的位置，再与x做“|”运算，其他部分不变只有第n位被改为1
- 当t = 0时，如法炮制，唯一不同的是“|”运算变成了“&”运算，这样就保证了该位置一定是0

当然，考虑到需要输出两次二进制编码，为了降低重复性，我就额外写了一个函数

代码如下：

```
1  #include <stdio.h>
2
3  void print(int x);
4
5  int main(){
6      int x;
7      int n;
8      int t;
9      printf("请输入一个十进制数");
10     scanf("%d",&x);
11     printf("你要更改该数字的第几位: ");
12     scanf("%d",&n);
13     printf("你要更改的数字为 (0或1): ");
14     scanf("%d",&t);
15
16     printf("该数字原本的二进制为: ");
17     if(x == 0){
18         printf("0");
19     }else {
20         print(x);
21     }
22
23     //防止出现超位
24     if (n < 1 || n > 31) {
25         printf("\n你输入的位数超出范围");
26         return 1;
27     }
28     //对x进行更改
29     if(t == 1){
30         x |= (1 << (n-1));
31     }else {
32         x &= ~(1 << (n-1));
33     }
34
35     printf("\n更改后，该数字的二进制为: ");
36     print(x);
37
38     return 0;
39 }
40
41 void print(int x){
42     int started = 0;
43
44     for (int i = 31; i >= 0; i--) {
45         //从第一个1开始输出
46         if (x & (1 << i)) {
```

```

47         started = 1;
48     }
49     if (started) {
50         printf("%d", (x & (1 << i)) ? 1 : 0);
51     }
52 }
53 return;
54 }

```

运行结果如图：

```

5  int main(){
12  scanf("%u",&n);
13  printf("你要更改的数字为(0或1):");
14  scanf("%d",&t);
15
16  printf("该数字原本的二进制为: ");
17  if(x == 0){
18      printf("0");
19  }else {
20      print(x);
21  }
22
23  //防止出现超位
24  if (n < 1 || n > 31) {
25      printf("\n你输入的位数超出范围");
26      return 1;
27  }
28  //对x进行更改
29  if(t == 1){
30      x |= (1 << (n-1));
31  }else {
32      x &= ~(1 << (n-1));
33  }
34
35  printf("\n更改后,该数字的二进制为:");
36  print(x);

```

问题 输出 调试控制台 终端 端口

```

PS D:\c\vscode\vscode> ${env:DEBUGINFOD_URLS}='' & 'c:\Users\Mark\vscode\exte
Microsoft Windows [版本 10.0.17134.1] (x64; 中文(简体); Windows 10)
(c) 2018 Microsoft Corporation. 保留所有权利。
C:\Users\Mark>
C:\Users\Mark> cd D:\c\vscode\vscode
C:\Users\Mark> gcc 1.c
C:\Users\Mark> .\1.exe
请输入一个十进制数251
你要更改该数字的第几位: 3
你要更改的数字为(0或1): 1
该数字原本的二进制为: 11111011
更改后, 该数字的二进制为: 11111111

```

4.

之前偶然间了解过这个算法的我，做这个题，感觉有点作弊啊...

嗯，在二进制的世界里，一个整数和它的相反数有着明显的形式上的联系：它们最右边1位数相同，以此为节点，高位上的数字完全相反，低位上的数字都是0（感觉我说了句废话...）

那就不难想到：将这两个数做“&”运算，就只剩下最右边的1，而其他位都为0了，然后再通过移位遍历，充斥疑云的天空就豁然开朗了

```
1 #include <stdio.h>
```

```

2
3 int main (){
4     int x;
5     printf("请输入一个有符号的十进制整数: ");
6     scanf("%d",&x);
7     int y = x; //引入y来存储x的初始值
8     int z = 0; //存储第一个1的位置
9     x &= -x;
10
11     for(int i = 0; i < 31; i++){
12         if(x & (1 << i)){
13             z = i+1;
14
15         }
16     }
17
18     if(z){
19         printf("该数字的第一个1在第%d位",z);
20     }else {
21         printf("二进制表示中没有1! ");
22     }
23
24     printf("\n该十进制数的二进制表示为: ");
25     if(y == 0){
26         printf("0");
27     }else {
28         int started = 0;
29
30         for (int i = 31; i >= 0; i--) {
31             if (y & (1 << i)) {
32                 started = 1;
33             }
34             if (started) {
35                 printf("%d", (y & (1 << i)) ? 1 : 0);
36             }
37         }
38     }
39     return 0;
40 }

```

运行结果如下:

问题 输出 调试控制台 终端 端口

5.

所以，只有 a 和 b 完全相同的高位才会保留下来，换句话讲，只需要对a, b同时做 ">>" 运算直至a, b 高位相同即可

在尝试了多种方法之后，我想：算了，毁灭吧，就这样吧...

如果有标答，麻烦出题人留言一下，感谢...

```

1  #include <stdio.h>
2
3  int main() {
4      char a[101], b[101];
5      printf("请输入二进制编码a:");
6      scanf("%s", a);
7      printf("请输入二进制编码b:");
8      scanf("%s", b);
9      //计算a,b的位数
10     int len_a = 0, len_b = 0;
11     while (a[len_a]) len_a++;
12     while (b[len_b]) len_b++;
13     //如果a, b的位数不相同, 那么 & 的运算结果肯定为0
14     if(len_b != len_a){
15         for(int i = 0; i < len_b; i++){
16             printf("0");
17         }
18     }else {
19         int count;
20         for(count = 0; b[count] && a[count] == b[count]; count++){
21
22             for (int k = 0; k < len_b; k++){
23                 printf("%c", k < count ? b[k] : '0');
24             }
25         }
26
27         return 0;
28     }

```

运行结果如图：

```

1  #include <stdio.h>
2
3  int main() {
4      char a[101], b[101];
5      printf("请输入二进制编码a:");
6      scanf("%s",a);
7      printf("请输入二进制编码b:");
8      scanf("%s",b);
9      //计算a,b的位数
10     int len_a = 0, len_b = 0;
11     while (a[len_a]) len_a++;
12     while (b[len_b]) len_b++;
13     //如果a, b的位数不相同, 那么 & 的运算结果肯定为0
14     if(len_b != len_a){
15         for(int i = 0; i < len_b; i++){
16             printf("0");
17         }
18     }else {
19         int count;
20         for(count = 0; b[count] && a[count] == b[count]; count++);
21
22         for (int k = 0; k < len_b; k++){
23             printf("%c", k < count ? b[k] : '0');
24         }
25     }
26
27     return 0;

```

问题 输出 调试控制台 终端 端口

```

.cpptools-1.34.4-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microso
-In-xmy0qslf.2ft' '--stdout=Microsoft-MIEngine-Out-m1jh5wae.3ad' '--stderr=Microsoft-MI
...
-interpreter=mi'
请输入二进制编码a:10010
请输入二进制编码b:10100
10000

```

思考题

1.

(1) 区别

- 运算对象：
 - 逻辑运算的运算对象是逻辑运算符两边的布尔值或条件表达式
 - 位运算的运算对象是数，变量，计算式所表示的整数类型的二进制表示
- 运算法则：
 - 逻辑运算的运算法则主要是“与、或、非”
 - 位运算的运算法则除了“&”、“|”，还有“按位异或”（^）、“按位取反”（~）、“左移”（<<）、“右移”（>>）：总的来说，种类更多，更加精细
- 运算结果：
 - 逻辑运算的运算结果只有两种状态：真（1）、假（0）
 - 位运算的运算结果是对二进制位逐位操作后得到的新数值
- 另外，逻辑运算具有短路特性

(2) 联系

- 二者的运算逻辑具有相似性：“&”和逻辑“&&”都遵循“全1才为1，有0则为0”的规则；位或“|”和逻辑或“||”都遵循“有1则为1，全0才为0”的规则
- 逻辑运算和位运算在逐位运算时返回值都是0或1

2.位运算的性质

(1) 基础代数性质

- 交换律，例如： $x \& y = y \& x$ ， $x \wedge y = y \wedge x$
- 结合律，例如： $(x \& y) \& z = x \& (y \& z)$ ， $(x \wedge y) \wedge z = x \wedge (y \wedge z)$
- 幂等律（自身与自身进行“与”或“或”运算，结果不变），例如 $x \& x = x$ ， $x | x = x$ （PS：“按位异或”运算不适用此定律）
- 分配律，例如：
 - “按位与”对“按位或/异或”分配： $x \& (y | z) = (x \& y) | (x \& z)$ ， $x \& (y \wedge z) = (x \& y) \wedge (x \& z)$
 - “按位或”对“按位与”分配： $x | (y \& z) = (x | y) \& (x | z)$
 （PS：异或不分配于与或，或也不分配于异或）
- 德·摩根律（“按位取反”可以穿透“按位与/或”运算并改变运算符），即 $\sim(x \& y) = (\sim x) | (\sim y)$ ， $\sim(x | y) = (\sim x) \& (\sim y)$

(2) 特殊运算符性质

- 按位异或（ $\wedge$ ）的特殊性质：
 - 任何数与0异或等于其本身： $x \wedge 0 = x$ 。
 - 任何数与自身异或等于0： $x \wedge x = 0$ 。
 - 异或运算是可逆的，不会造成信息丢失，常用于无中间变量交换两个数或简单的加密解密。
- 按位取反（ $\sim$ ）的性质：
 - 在计算机补码机制下，取反运算满足公式： $\sim x = -(x + 1)$
- 移位运算（ $\ll, \gg$ ）的性质：
 - 左移（ $\ll$ ）：等价于乘以2的k次方（ $x \ll k = x * 2^k$ ），低位补0，高位丢弃（可能溢出）。
 - 右移（ $\gg$ ）：等价于除以2的k次方并向下取整（ $x \gg k = \text{floor}(x / 2^k)$ ）。对于负数，算术右移高位补1；对于正数，高位补0。
 - 不可逆性：移位运算由于边缘位的丢弃，不具备逆运算

(3) 运算优先级

- 算术运算符（+，-，*，/）> 移位运算符（ $\ll, \gg$ ）> 按位与（&）> 按位异或（ $\wedge$ ）> 按位或（|）

3.

(1) 补码

为了统一加减法运算并简化硬件设计，采用了补码作为有符号整数的唯一存储与运算编码（没错，这句话是我copy的 [端手]）

说人话就是：我们需要遵循“一个数与它的相反数之和为0”这一规则，比如 $1 + (-1) = 0$ ；如果1的二进制编码是0001，0的二进制编码是0000，那么-1的二进制编码就应该是1111，而“1111”也叫做-1的补码

所以，补码的数学原理也显而易见了：在n位二进制系统中，模为 2^n ，负数的补码实际上等于 2^n 减去该数的绝对值

并且，由此可知，一个4位二进制系统在无符号数时，能够存储的数为[0, 15]；在存储有符号数时，能够存储的数的范围是[-8, 7]，**负数**和**非负数**刚好一半一半，而首位为0和首位为1刚好各自可以表示期间内一半的数！于是，很轻松能想到负数的首位应当是1，而正数的首位应当是0，这也就是我们把最高位称作“符号位”的原因了

(2) 右移

逻辑右移的基本规则就是低位丢弃，高位补0，而“ $\gg n$ ”的数学本质就是将原十进制数除以 2^n ，并向下取整

这看似很合理，但当我们把目光放到负数上就会发现：当我们在高位补零的时候，理应为负的结果变成了正数，例如： $(-8)/2$ 结果应该等于-4，但是在二进制系统中进行逻辑右移，就会有： $1000 \gg 1$ 得到0100即4！有点让人啼笑皆非

于是就有了：

算数右移：将各位依次向右移动，低位丢弃，高位用原符号位（最高位）进行填充（即符号扩展），并向下取整

这样一来，当对-8进行算数右移1时，就会得到1100，即-4，丸美！

模块三：通信模块 —— 无碰撞协议

【任务3】编程题：字符串交集检测

整型的外在表现是一个十进制数，如何让它的每一个字节都存储信息就是首先需要解决的问题。我们知道每当这个十进制数改变的时候，与之对应的32位字节也会发生改变，于是，我开始考虑通过十进制数间接映射每一个字节的存储状态

随即联想到两点：

- 一是：十进制数的加法与二进制并无本质不同，也就是说，想要第一位（最低位）为1，十进制数就需要加 $2^0 = 1$ ；要想第三位为1，就需要十进制加 $2^2 = 4$
- 二是：字符之间也能够进行运算（我之前还老是笑字母之间比较大小有什么意义，真是“存在即合理”啊），比如说 $'a' - 'a' = 0$ ， $'b' - 'a' = 1$ ，嗯，难怪题目非要强调是小写字母呢！

基于这两点，问题不就迎刃而解了吗？于是，你将看到我的初代循环：

```
1  for(int i = 0; s1[i]; i++){
2      mask1 += s1[i] - 'a' + 1;
3  }
```

写完之后我才发现题目的要求是：“请利用位运算”，好吧好吧...

在这之后我思考了很久，如何才能用位运算改变32位整型的值呢？

既然不能直接在十进制数上做文章，那何必舍近求远、曲线救国呢？于是我直接对整型的二进制编码动手了：保留了原来的字符运算，将1左移后与mask做“按位或”运算（只要该位上有1，就为1，即只要字母出现过，该位就记为1），真是太有思路了！

于是，代码就长这样了：

```
1  #include <stdio.h>
2
3  int hasCommonChar(const char *s1, const char *s2) {
4      int mask1 = 0;
5      int mask2 = 0;
6  }
```

```

7     for(int i = 0; s1[i]; i++){
8         mask1 |= (1 << (s1[i] - 'a'));
9
10    }
11    for(int i = 0; s2[i]; i++){
12        mask2 |= (1 << (s2[i] - 'a'));
13    }
14
15    //当两个字符串没有公共字符时，返回0，否则返回1
16    return (mask1 & mask2) != 0;
17 }
18
19 int main() {
20     printf("%d\n", hasCommonChar("hello", "world"));
21     printf("%d\n", hasCommonChar("abc", "xyz"));
22     return 0;
23 }

```

```

1  #include <stdio.h>
2
3  int hasCommonChar(const char *s1, const char *s2) {
4      int mask1 = 0;
5      int mask2 = 0;
6
7      for(int i = 0; s1[i]; i++){
8          mask1 |= (1 << (s1[i] - 'a'));
9
10     }
11     for(int i = 0; s2[i]; i++){
12         mask2 |= (1 << (s2[i] - 'a'));
13     }
14
15     //当两个字符串没有公共字符时，返回0，否则返回1
16     return (mask1 & mask2) != 0;
17 }
18
19 int main() {
20     printf("%d\n", hasCommonChar("hello", "world"));
21     printf("%d\n", hasCommonChar("abc", "xyz"));
22     return 0;
23 }

```

问题 输出 调试控制台 终端 端口

```

● PS D:\c\.vscode\.vscode> ${env:DEBUGINFOD_URLS}=''; & 'c:\Users\
-stdin=Microsoft-MIEngine-In-0nmujjfv.kmy' '--stdout=Microsoft-MI
0i3unt.5un' '--dbgExe=D:\c\mingw64\bin\gdb.exe' '--interpreter=mi
1
0

```

模块四：反应堆核心 —— 绝对平衡的艺术

【任务 4】

必须要明确的是：一个 $N \times N$ 的二维矩阵，每一行最多只能放 1 根，每一列最多也只能放 1 根。当放入 N 根控制棒时，这 N 行和 N 列已经占满了，于是可知 $N \times N$ 的二维矩阵最多可以放 N 根控制棒（当然， $N = 2$ 、 $N = 3$ 时是不会出现解的，这是我做归纳法时发现的）

- 首先，必须要解决的问题是：如何将二进制编码和矩阵联系起来。不难想到，每一串二进制编码都可以看作行为 1，列为 N 的矩阵，在对 N 个它们进行位运算的时候，就会组成 $N \times N$ 的二维矩阵
- 其次，亟待解决的是如何检测行和斜线方向上是否有超过一根“控制棒”
 - 在行方向上：根据常识，二进制编码的第 n 位可以表示为 2^{n-1} ，所以，只需要检测该十进制数能否被 2^{n-1} 整除即可（然而，这样的检测方法在我写代码的过程中已经淘汰掉了）
 - 在斜线方向上：联想到位运算中的“位移”，可以依次做 “ $<<1$ ” “ $>>1$ ” 运算，然后进行 “&” 运算
- 再次，还需要解决的问题是如何遍历出所有方案
 - 我首先想到的是双层 for 循环，然而，其时间复杂度为 $O(n^2)$ ，唉，可惜了可惜了...
 - 在这之后，我又想到了在学习二叉树时所使用的递归，然而，纯递归又会和上面的双层 for 一样，怎么办呢...
 - 遇事不决问 AI，AI 给了我一个数组版的解决“N 皇后问题”的遍历所有方案的代码：

```
1 // 递归函数：row 代表当前正在处理第几行
2 void solve(int row) {
3     // 递归出口：如果 N 行都处理完了，说明找到了一个合法方案
4     if (row == N) {
5         printBoard();
6         return;
7     }
8
9     // 【单层 for 循环】：在当前 row 行中，依次尝试 0 到 N-1 列
10    for (int col = 0; col < N; col++) {
11        if (isSafe(row, col)) { // 如果这个位置安全
12            board[row][col] = 1; // 1. 放入控制棒
13            solve(row + 1);      // 2. 【递归】去下一行继续放
14            board[row][col] = 0; // 3. 【回溯】拿掉控制棒，尝试当前行
15                                // 的下一列
16        }
17    }
```

说实话，`board[row][col] = 0;`这句话我还理解了半天[擦汗]

- 正所谓：他山之石，可以攻玉。这下好了好了这下好了，就只剩最后一个问题了：规定存储控制棒的数据形式
 - 要想一串二进制编码中只有一个 1，就只需要其对应的十进制数是 2 的次幂，也就是说，仅额外添加一个幂函数，问题就迎刃而解了，但好巧不巧，问题就出在幂函数上：它的时间复杂度为 $O(n^2)$ ！**Fail!**
 - 好吧，想想其他办法吧，有道是：七步之内必有解药。不如就从二进制编码中寻找答案...
假设这个十进制数为 x ，如果 x 对应二进制表示只有一个 1，那么 $x-1$ 的二进制表示就是，对应位置 1 变成 0，后面 0 全部变成 1；将 $x-1$ 与 x 做 “&” 运算，只要 x 二进制表示里不止一个 1，结果必然不为 0！

然而，戏剧的是，在我绞尽脑汁，思考清楚这一切之后，突然灵感光一闪：我为什么不直接对1进行“<<”呢！这集太有喜剧效果

所以，代码最后就长这样：

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int is_safe(int row[],int r);//r表示当前行数
5  void solve(int row[],int N,int r,int *count);
6  void print(int row[],int N);
7
8  int main(){
9      int N;
10     printf("请给定N的值: ");
11     scanf("%d",&N);
12     if(N > 0 && N <=15){
13         int row[N];
14         int x = 0;
15         int count = 0;
16
17         printf("可能的方案有: \n");
18         solve(row,N,x,&count);
19
20         printf("总共有%d种方案",count);
21     }else {
22         printf("错误! ");
23     }
24     return 0;
25 }
26
27 //检测插入的控制棒是否安全
28 int is_safe(int row[],int r){
29     if (r == 0) return 1;
30
31     //检查正上方的列
32     for(int i = 0;i < r;i++){
33         if ((row[r] & row[i]) != 0) {
34             return 0;
35         }
36
37         //行距是r-i，只需要将row[i]左移或右移(r-i)位后做&就可以了
38         //检查右上方斜线
39         if(((row[i] << (r - i)) & row[r]) != 0){
40             return 0;
41         }
42
43         //检查左上方斜线
44         if(((row[i] >> (r - i)) & row[r]) != 0){
45             return 0;
46         }
47     }
48
49     return 1;
50 }
51
52 //打印所有可行的方案
```

```

53 void print(int row[], int N){
54     for(int i = 0; i < N; i++){
55         for(int j = N - 1; j >= 0; j--){
56             // 有控制棒的位置打印 1, 没有的位置打印 0
57             printf("%d ", (row[i] & (1 << j)) ? 1 : 0);
58         }
59         printf("\n");
60     }
61     printf("-----\n");
62 }
63
64 //生成所有可行的方案
65 void solve(int row[],int N,int r,int *count){
66     //递归出口
67     if(r == N){
68         (*count)++;
69         print(row,N);
70         return;
71     }
72
73     for (int index = 0;index < N;index++){
74         row[r] = (1 << index);
75
76         if(is_safe(row,r)){
77             solve(row,N,r+1,count);
78         }
79     }
80     return;
81 }
82 ...

```

运行结果如图：

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int is_safe(int row[],int r);//r表示当前行数
5  void solve(int row[],int N,int r,int *count);
6  void print(int row[],int N);
7
8  int main(){
9      int N;
10     printf("请给定N的值:");
11     scanf("%d",&N);
12     if(N > 0 && N <=15){
13         int row[N];
14         int x = 0;
15         int count = 0;
16
17         printf("可能的方案有: \n");
18         solve(row,N,x,&count);
19
20         printf("总共有%d种方案",count);
21     }else {
22         printf("错误! ");
23     }
24     return 0;
25 }
26
27 //检测插入的控制棒是否安全
28 int is_safe(int row[],int r){
29

```

问题 输出 调试控制台 终端 端口

```

PS D:\c\.vscode\.vscode> ${env:DEBUGINFOD_URLS}='' & 'c:\Users\Mark\.vscode\In-nvrby223.phr' '--stdout=Microsoft-MIEngine-Out-51us2b11.1c3' '--stderr=
nterpreter=mi'
请给定N的值: 4
可能的方案有:
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
-----
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
-----
总共有2种方案

```

但是，我隐隐约约有一种感觉，这只是我的极限而非答案的极限，毕竟我没有将时间复杂度压缩到极限，于是，我在B站上找到了“n皇后问题”的位运算教程，学习之后，自己写了plus版：

```
1 #include <stdio.h>
```

```

2
3  int N;
4
5  int solve(int col,int left,int right);
6
7  int main(){
8      printf("请给定N的值: ");
9      scanf("%d",&N);
10
11     if (N < 1 || N > 15){
12         printf("错误! ");
13         return 0;
14     }
15     //列信息
16     int col = 0;
17     //左下信息
18     int left = 0;
19     //右下信息
20     int right = 0;
21     printf("共有%d种方案",solve(col,left,right));
22
23     return 0;
24 }
25
26 int solve(int col,int left,int right){
27     if(N < 1){
28         return 0;
29     }
30     int limit = (1 << N) - 1;
31     if(col == limit){
32         return 1;
33     }
34     //1表示可以放控制棒, 0表示不行, 这里取反的原因, 我认为是为了和limit保持一致
35     int chance = limit & ~(col | left | right));
36     //存储最低位1
37     int place = 0;
38     //计数变量
39     int count = 0;
40
41     while(chance != 0){
42         place = (chance & (-chance));
43         //消去最低位1, 表示已经试过了
44         chance ^= place;
45         //注意符号优先级, 我第一次运行的时候N=4, 给我干了8种出来了(哭笑)
46         count += solve((col | place),(left | place) >> 1,(right | place) <<
1);
47     }
48
49     return count;
50 }

```

如果要输出每一种可能, 就必须要用到数组, 所以我索性没有这种加上更直观部分

在我看来, 这种算法之所以快, 是因为完全抛弃了物理层面的二维矩阵, 换句话说, 其实在代码运行过程中, 根本就没有所谓的“列”, 有的只是记录状态的二进制编码, 尤其是while循环, 让人忍不住感叹: 增之一分则太长, 减之一分则太短, 每一句都是精华啊!

这种纯粹的靠逻辑来改变数据状态, 使之同时存储过去和现在信息的方法, 给了我很大的震撼和启发

以下是运行结果

```
7   int main(){
11      if (N < 1 || N > 15){
12          return 0;
13      }
14      int col = 0;
15      int left = 0;
16      int right = 0;
17      printf("共有%d种方案",solve(col,left,right));
18
19      return 0;
20  }
21
22
23  int solve(int col,int left,int right){
24      if(N < 1){
25          return 0;
26      }
27      int limit = (1 << N) - 1;
28      if(col == limit){
29          return 1;
30      }
31
32      int chance = limit & (~(col | left | right));
33      int place = 0;
34      int count = 0;
35
36      while(chance != 0){
37          place = (chance & (-chance));
38          chance ^= place;
39          count += solve((col | place),(left | place) >> 1,(right | place) << 1);
40      }
41
42      return count;
43  }
```

问题 输出 调试控制台 终端 端口

```
PS D:\c\vscode\vscode> ${env:DEBUGINFOD_URLS}='' & 'c:\Users\Mark\vscode\extensions\ms-vscode
e=D:\c\mingw64\bin\gdb.exe' '--interpreter=mi'
请给定N的值: 15
共有2279184种方案
PS D:\c\vscode\vscode> ^C
PS D:\c\vscode\vscode> ${env:DEBUGINFOD_URLS}='' & 'c:\Users\Mark\vscode\extensions\ms-vscode
rosoft-MIEngine-In-00a4u0sv.cym' '--stdout=Microsoft-MIEngine-Out-mwluqot1.gcb' '--stderr=Microso
e=D:\c\mingw64\bin\gdb.exe' '--interpreter=mi'
请给定N的值: 4
共有2种方案
```

【任务 5】

在做这个题的时候，有很多想法，颇有一种“剪不断，理还乱”的感觉，思路调整了很多遍，一开始，我准备用递归，但是很快发现：没办法找到最短路径，除非加计数变量，不仅如此，递归又要写函数，又要传指针，又要比大小，算了算了...泥肘吧

既然递归不行，那只能用循环了，循环的思路还算是清晰（如下图），但是敲代码的时候出现了很大的困难，比如说：对于循环终止的判定语句（我反复改了三四遍），以及数组越界（老是Segmentation fault）等等，嗯，总的来说，最困难的应该还是“出队”“入队”（这道题的删除队列的方式其实是frount++）以及移动M.O.U.S.E.移动的判定吧（说实话，我到现在都还不是很适应这种矩阵的形式，虽然算上这次我已经接触过三次了），我知道判断是否撞墙的if语句写的有点丑陋，见谅见谅，呵呵...

```

✓ /*确定方向:
   上 left = 4*dr[0] + dc[0];
   下 right = 4*dr[1] + dc[1];
   左 down = 4*dr[2] + dc[2];
   右 up = 4*dr[3] + dc[3];
*/

✓ //循环结束条件: 有最优解: pos == 15;无解: front >= rear
//走一步:

✓ /*确定方向 (pos += 方向)

判断格子编号是否在 0~15
判断能不能走 (边界/墙/已经过)
pos < 4 :no up
pos > 11 : no down
pos % 4 == 0 : no left
pos % 4 == 3 :no right
(visited & (1 << pos)输出0就说明可以走)
*/

//更新状态: visited |= (1 << pos),dist++;

//传入新的数据结构

//END OF TO DO

```

```

1  #include <stdio.h>
2  #include <stdbool.h>
3
4  // 定义简单队列结构用于 BFS
5  typedef struct {
6      int pos;    // 当前节点编号 (0~15)
7      int dist;   // 到达当前节点的最短步数
8  } Node;
9
10 int minStepsToCheese(int walls) {
11     int start = 0;
12     int target = 15;
13
14     // 如果起点或终点本身是墙，直接不可达
15     if ((walls & (1 << start)) || (walls & (1 << target))) {
16         return -1;
17     }
18
19     // BFS 队列与访问位图
20     Node queue[16];
21     int front = 0, rear = 0;
22     int visited = 0;
23
24     // 起点入队并标记已访问 (请使用位运算)
25     queue[rear++] = (Node){start, 0};
26     visited |= (1 << start);
27
28     // 上、下、左、右四个方向的节点偏移量

```

```

29     int dr[4] = {-1, 1, 0, 0};
30     int dc[4] = {0, 0, -1, 1};
31
32     //请在TO DO 和END OF TO DO 行之间补全代码:
33     //TO DO
34     while(front < rear){
35         Node p = queue[front++];
36
37         if(p.pos == 15){
38             return p.dist;
39         }
40
41         for(int i = 0;i < 4;i++){
42             Node next_p;
43
44             next_p.pos = p.pos + 4*dr[i] + dc[i];
45             next_p.dist = p.dist + 1;
46
47             if (next_p.pos < 0 || next_p.pos > 15) continue;
48
49             if ((i == 0 && p.pos < 4) ||
50                 (i == 1 && p.pos > 11) ||
51                 (i == 2 && p.pos % 4 == 0) ||
52                 (i == 3 && p.pos % 4 == 3) ||
53                 (walls & (1 << next_p.pos)) ||
54                 (visited & (1 << next_p.pos)))
55             {
56                 continue;
57             }
58
59             visited |= (1 << next_p.pos);
60             queue[rear++] = (Node){next_p.pos,next_p.dist};
61         }
62     }
63
64
65     /*确定方向:
66     上 left = 4*dr[0] + dc[0];
67     下 right = 4*dr[1] + dc[1];
68     左 down = 4*dr[2] + dc[2];
69     右 up = 4*dr[3] + dc[3];
70     */
71
72     //循环结束条件: 有最优解: pos == 15;无解: front >= rear
73     //走一步:
74
75     /*确定方向 (pos += 方向)
76
77     判断格子编号是否在 0~15
78     判断能不能走 (边界/墙/已经过)
79     pos > 11 : no down
80     pos < 4 :no up
81     pos % 4 == 0 : no left
82     pos % 4 == 3 :no right
83     (visited & (1 << pos)输出0就说明可以走)
84     */
85
86     //更新状态: visited |= (1 << pos),dist++;

```

```

87
88 //传入新的数据结构
89
90 //END OF TO DO
91
92 return -1; // 无法到达
93 }
94
95 int main() {
96     int walls = (1 << 5) | (1 << 10); // 5号和10号格子是墙
97     int steps = minStepsToCheese(walls);
98     printf("Minimum steps: %d\n", steps); // 应输出 6
99     return 0;
100 }

```

运行结果如下：

```

10  int minStepsToCheese(int walls) {
23
24     // 起点入队并标记已访问 (请使用位运算)
25     queue[rear++] = (Node){start, 0};
26     visited |= (1 << start);
27
28     // 上、下、左、右四个方向的节点偏移量
29     int dr[4] = {-1, 1, 0, 0};
30     int dc[4] = {0, 0, -1, 1};
31
32     //请在TO DO 和END OF TO DO 行之间补全代码:
33     //TO DO
34     while(front < rear){
35         Node p = queue[front++];
36
37         if(p.pos == 15){
38             return p.dist;
39         }
40
41         for(int i = 0; i < 4; i++){
42             Node next_p;
43
44             next_p.pos = p.pos + 4*dr[i] + dc[i];
45             next_p.dist = p.dist + 1;
46
47             if (next_p.pos < 0 || next_p.pos > 15) continue;
48
49             if ((i == 0 && p.pos < 4) ||
50                 (i == 1 && p.pos > 11) ||
51                 (i == 2 && p.pos % 4 == 0) ||
52                 (i == 3 && p.pos % 4 == 3) ||
53                 (walls & (1 << next_p.pos)) ||
54                 (visited & (1 << next_p.pos)))
55             {
56                 continue;
57             }
58
59             visited |= (1 << next_p.pos);
60             queue[rear++] = (Node){next_p.pos, next_p.dist};
61         }
62     }

```

问题 输出 调试控制台 终端 端口

...

Minimum steps: 6

后记

写完代码，虽然程序跑通了，但我总觉得对“广度优先搜索（BFS）”的理解还不够透彻，并没有get到其本质。于是我又查阅了一些有关BFS的资料，顺带的把还有DFS

这两种算法都是图论的基石（虽然我现在还没有深入了解图论），DFS算是老朋友了，它优先深度，一条路走到底，靠递归来实现，回想之前在“树”那节，我还被狠狠拷打了...

BFS则与之相反，它优先广度，通过队列来实现，相当于按层往外推进。不过有个细节曾困扰我很久：在每一次大循环里，单源的遍历是有先后顺序的，这和“同时搜索一层”的思路似乎有点矛盾

但是，其实“按层搜索”是它的本质（或者说是目的），而“顺序遍历可行的位置”则是外在表现，这种表现形式是由队列存储数据的特点决定的：正是因为队列保证了先进先出，同一层的节点才会被依次处理，从而在宏观上呈现出“一层一层往外扩”的效果

除此之外，我还在了解更多，说不定当学长你看到这段文字的时候，我正在学习多源BFS，01BFS呢...